Q2: Image Colourization using CNNs (75 Marks)

Overview

In this task, you will implement and train a convolutional encoder–decoder network for **image colourization** on the CIFAR-10 dataset (32×32 RGB images). All code must be written in **PyTorch**, and you must log training, validation, and hyperparameter sweeps using **Weights & Biases (wandb)**.

Given a grayscale input image, your goal is to predict the colour of each pixel, classifying it into one of 24 colour clusters.

Colour Centroids and Data Preparation

To simplify the image colourization task, colour prediction is treated as a **classification** problem rather than regression. Each RGB pixel in CIFAR-10 is assigned to one of 24 representative clusters (colour centroids).

Provided: `color_centroids.npy` (shape: [24,3]) — 24 representative RGB cluster centroids computed by applying k-means ($k = 24$) over CIFAR-10 training pixels.

- Each pixel (x, y) is mapped to the nearest centroid $\mathbf{c}_i$:

$$\text{label}(x, y) = \arg \min_{i \in \{1,\ldots,24\}} \|\text{RGB}(x, y) - \mathbf{c}_i\|_2$$

- This produces a class map of size [32,32] with values in $\{0, \ldots, 23\}$.
- The model input is a grayscale image [1,32,32].
- Model output is logits per pixel [24,32,32].
- The loss is per-pixel cross-entropy.

Model Architecture: (20 pts)

In this architecture, the encoder is designed to **progressively reduce the spatial dimensions** of the input image through multiple convolutional and pooling layers. This deeper compression enables the network to capture more abstract and spatially invariant features. The decoder, on the other hand, reconstructs the colourized output image by performing **learned upsampling** using `nn.ConvTranspose2d` (transpose convolution) layers instead of simple interpolation. This approach allows the model to learn the most effective way to recover fine spatial details during reconstruction, encouraging students to understand how receptive fields expand in the encoder and how transpose convolutions can learn spatial mappings in the decoder.

Recommended block sequence

Use this architecture as the baseline. Experiments with kernel, NIC, NF.

```
Input [B, NIC, 32, 32]
Conv2d -> BN -> ReLU -> MaxPool2d   -> [B, NF, 16, 16]
Conv2d -> BN -> ReLU -> MaxPool2d   -> [B, 2*NF, 8, 8]
Conv2d -> BN -> ReLU -> MaxPool2d   -> [B, 4*NF, 4, 4]    # extra downsample
ConvTranspose2d -> BN -> ReLU       -> [B, 2*NF, 8, 8]   # learned upsample
ConvTranspose2d -> BN -> ReLU       -> [B, NF, 16, 16]
ConvTranspose2d -> BN -> ReLU       -> [B, NC, 32, 32]
Conv2d (classifier)                  -> [B, NC, 32, 32]
```

Architecture diagram

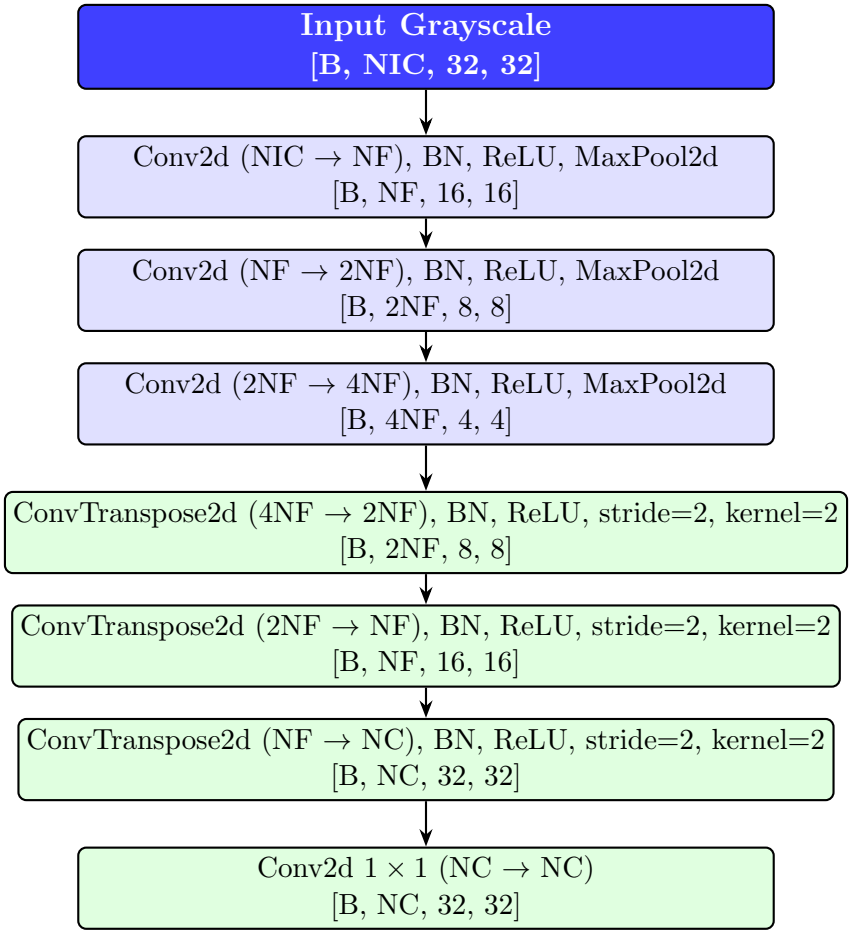


Figure 1: Model architecture

Training and Evaluation (20 pts)

- Dataset: CIFAR-10. Use supplied `color_centroids.npy` to map RGB images to 24-class labels; input is grayscale.

- Loss: per-pixel cross-entropy.
- Optimizer: SGD or Adam (log choice and hyperparameters in wandb).
- Train for at least 25 epochs; save and log the best checkpoint (lowest validation loss).
- At the end, produce:
    1. 10 example colourizations (input gray, predicted colourized, ground-truth colour).
    2. Training and validation loss curves.

Analytical Questions (10 pts)

For your `Model Architecture` compute (symbolically in terms of NIC, NF, NC):

1. Number of weights (ignore biases and BatchNorm parameters).
2. Number of outputs (total activation elements across all layers for a single input).
3. Number of connections (sum over layers of weight count times number of input activation elements each weight connects to).

Repeat the three quantities for input spatial size doubled (64×64). Show each step and state assumptions (padding/stride/kernels).

Hyperparameter Tuning (25 pts)

Perform a wandb sweep exploring at least 20 configurations. Tune at minimum:

- Learning rate: {1e-4, 3e-4, 1e-3, 3e-3}.
- Batch size: $\{32, 64, 128\}$.
- Number of filters (NF): $\{8, 16, 32\}$.
- Kernel size: $\{3, 5\}$ (adjust padding).
- Optimizer: SGD / Adam.

For each run log:

- Hyperparameters, train/val loss curves, best validation loss, example images, and best checkpoint.
- After the sweep, report the best run (hyperparameters and validation loss) and include the wandb run URL in your writeup.